

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using ~~PySpark~~ RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

Duration : 1 Hour

Total Marks:50

Design Task

Code of Conduct:

*I **T. Niranjana Kumar (192473007)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

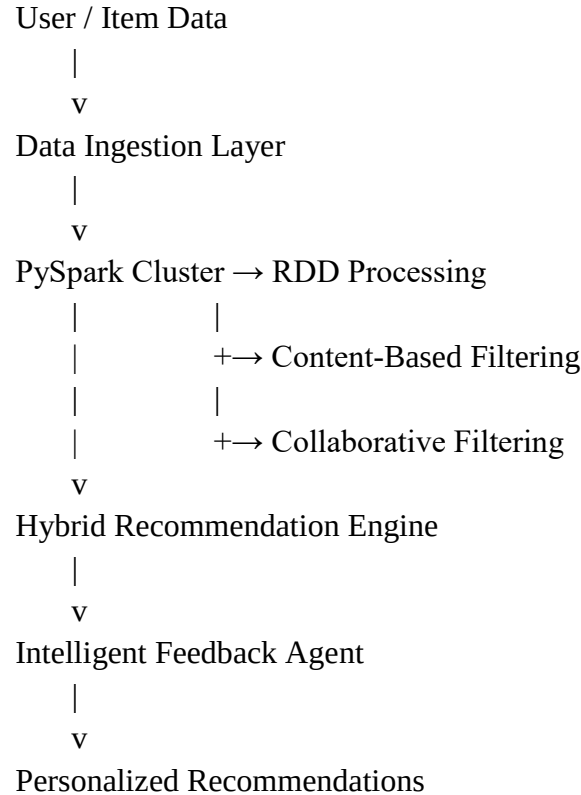
Signature of the student: T. Niranjana

1. HYBRID AI RECOMMENDATION ENGINE DESIGN

Problem Understanding & Objective

The system must recommend relevant items to users by integrating content-based filtering and collaborative filtering. PySpark RDDs are used to process large-scale user-item interactions in a distributed manner. Intelligent agents adapt recommendations using user behaviour and feedback in real time.

System Architecture



PySpark & RDD Operations

- `map()`: Extract user profiles, item features and ratings.
- `filter()`: Remove invalid or incomplete interactions.
- `join()`: Combine user interactions with item metadata.
- `reduceByKey()`: Aggregate ratings or interaction counts by user/item.
- `groupByKey()`: Group interactions for collaborative analysis where required.
- `collect()` / `take()`: Retrieve final results or samples for output.

Intelligent Agent Integration

A User Behaviour Agent observes clicks, views, ratings and feedback. A Recommendation Agent combines content similarity and collaborative scores. A Feedback Agent updates preference information after new interactions. Together, these agents continuously adapt the recommendation list.

Scalability & Distributed Design

RDD partitions allow user-item records to be processed in parallel across cluster nodes. The system can scale by adding worker nodes. RDD lineage and distributed execution support recovery from failures, while aggregation operations reduce large interaction data into manageable recommendation scores.

Data Flow

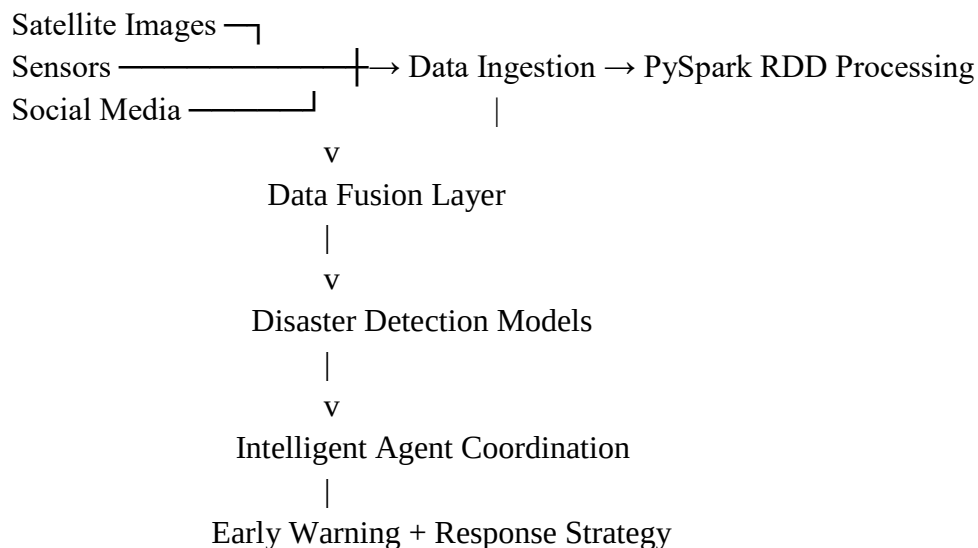
Interactions and item features → RDD creation → cleaning and transformation → parallel content and collaborative scoring → hybrid score combination → agent feedback update → personalized recommendations.

2. SMART DISASTER DETECTION SYSTEM

Problem Understanding & Objective

The system detects natural disasters such as floods or earthquakes from multi-source streams including satellite images, sensors and social media. The objective is early detection, coordinated warning generation and support for response strategies.

System Architecture



Data Fusion Techniques

- Feature-level fusion: Extract useful features from each source and combine them.
- Decision-level fusion: Combine alerts or predictions from separate detection components.
- Temporal and location alignment: Associate observations from different sources using time and geographic context.

RDD Operations

- `map()` for source-specific feature extraction.
- `filter()` for noisy or irrelevant records.
- `union()` to combine RDDs from multiple sources.
- `join()` to combine related records.
- `reduceByKey()` to aggregate evidence by region or event.

Intelligent Agents

A Sensor Agent monitors sensor observations, an Image Agent analyses satellite-derived features, a Social Monitoring Agent tracks relevant public signals, a Fusion Agent combines evidence, and an Alert Agent coordinates warning and response messages.

Scalability & Data Pipeline

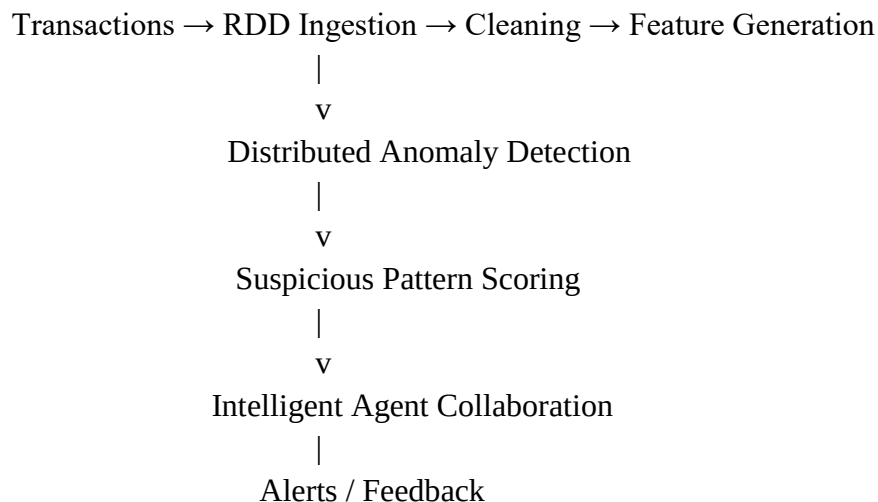
Each source can be processed in parallel using partitions. Aggregated regional evidence is passed to the detection layer. Distributed processing supports high data volume and allows the architecture to expand as more sensors, image feeds or social streams are added.

3. AI-BASED FRAUD DETECTION FRAMEWORK

Problem Understanding & Objective

The system processes large volumes of financial transactions using PySpark RDDs to identify suspicious patterns. Distributed anomaly detection and intelligent agents are used to trigger alerts and continuously improve detection accuracy.

Architecture & Pipeline



Distributed Anomaly Detection

Transaction records are partitioned across workers. Each partition calculates anomaly indicators such as unusual transaction amount, frequency or behavioural deviation. `reduceByKey()` can aggregate suspicious activity by account or region, while `join()` can combine transactions with historical profiles.

RDD Operations

- `map()`: Convert transactions into feature records.
- `filter()`: Remove invalid transactions or apply rule-based screening.
- `reduceByKey()`: Aggregate counts and anomaly evidence.
- `join()`: Combine current transactions with customer history.
- `distinct()`: Remove duplicate transaction identifiers where necessary.

Intelligent Agents

A Detection Agent scores suspicious activity, a Pattern Agent searches for repeated fraud behaviour, an Alert Agent sends high-risk notifications, and a Learning Agent uses feedback from verified cases to improve future detection.

Scalability

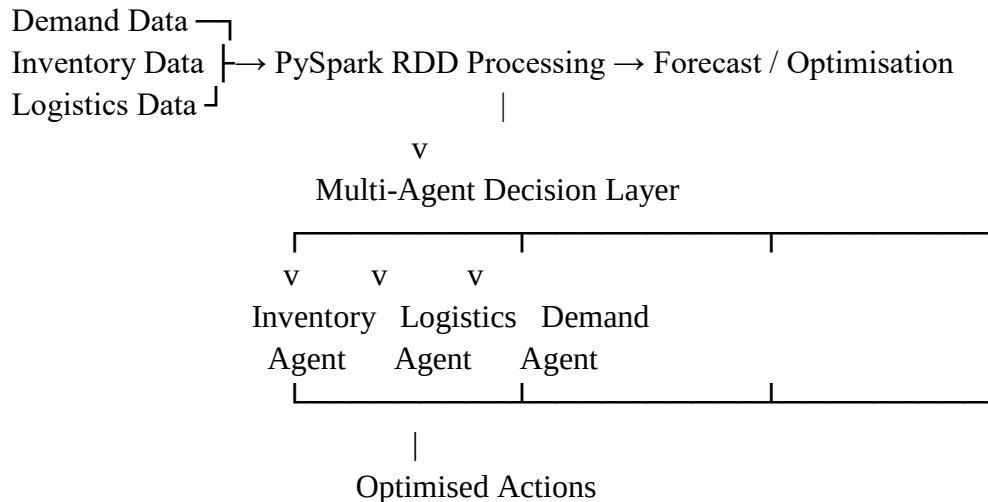
Large transaction volumes are divided across RDD partitions and processed in parallel. The design can scale horizontally by increasing workers. Distributed aggregation allows suspicious patterns to be detected across many partitions.

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

Problem Understanding & Objective

The system uses PySpark to optimise inventory, logistics and demand forecasting. Multiple intelligent agents make decentralised decisions to minimise cost and improve efficiency.

System Architecture



RDD Transformations & Actions

- `map()`: Transform raw demand, stock and logistics records.
- `filter()`: Select relevant products, routes or time periods.
- `join()`: Combine demand with inventory and logistics information.
- `reduceByKey()`: Aggregate demand and stock values by product/location.
- `sortByKey()`: Support priority ordering for selected optimisation results.

Intelligent Agent Interaction

The Demand Agent updates demand estimates, the Inventory Agent proposes replenishment decisions, and the Logistics Agent evaluates delivery options. Agents exchange current state information and coordinate decentralised decisions using shared optimisation objectives such as lower cost, reduced stock-outs and improved delivery efficiency.

Scalability & Fault Tolerance

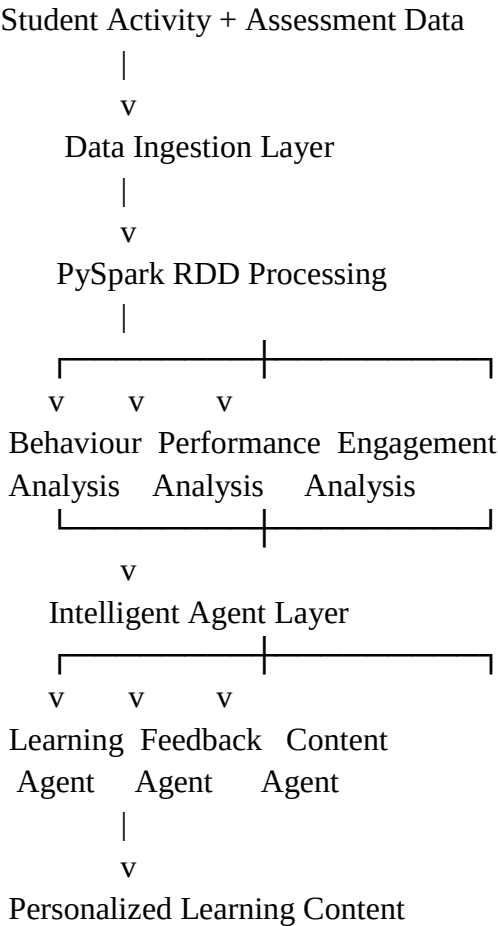
RDD partitions distribute large supply-chain records across workers. New nodes can be added as data volume grows. Distributed execution and RDD lineage support recovery and reliable processing.

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

Problem Understanding & Objective

The platform analyses student learning behaviour and performance data using PySpark and RDDs. It supports adaptive learning, real-time feedback and scalable analytics to personalize content delivery for thousands of learners.

System Architecture



RDD Operations

- map(): Convert activity logs into learner feature records.
- filter(): Remove incomplete or irrelevant events.
- reduceByKey(): Aggregate scores, attempts or activity by student.
- join(): Combine student profiles with assessment and content data.
- groupByKey(): Group multiple learning events for learner-level analysis.

Intelligent Agents

A Learning Agent estimates the learner's current needs, a Feedback Agent provides timely feedback based on performance, and a Content Agent selects suitable learning material. The agents use updated analytics to adapt content and recommendations.

Real-Time Feedback & Scalability

New learner events can be processed in distributed batches or integrated with a suitable streaming ingestion layer before RDD-based analysis. Parallel processing allows the platform to serve large numbers of learners while aggregating individual and overall learning analytics.

Innovation & Impact

The platform can combine performance, engagement and behaviour signals to provide more personalised learning paths. Agent coordination enables adaptation rather than a fixed one-size-fits-all content sequence.

FINAL CONCLUSION

The five designs demonstrate CO5 concepts by combining PySpark RDD transformations and actions, scalable distributed processing and intelligent-agent architectures. Each design defines a problem, system architecture, data flow, RDD operations, agent roles and scalability considerations, matching the assessment emphasis on distributed AI application design.